

**UNIVERSIDADE CESUMAR**

UNICESUMAR

AEP • 2º semestre de 2026

## **PSE em Ação**

*Sistema de Planejamento e Acompanhamento  
de Ações de Saúde na Escola*

### **Primeira entrega**

Análise e planejamento da solução

Tema de contextualização

Promoção da saúde mental no ambiente escolar

Curso e turma: ESOF2S-N-C

Integrante 1: Fernando Leme Origuela RA: 26008686-2

Integrante 2: Lucas Fontes Telles RA: 25194145-2

Integrante 3: Lucas Gabriel RA: 26014302-2

Maringá

2026

## 1 Contextualização

O Programa Saúde na Escola (PSE) é um instrumento que aproxima os serviços de saúde dos estudantes da educação básica pública, por meio de ações de prevenção, promoção e atenção à saúde (Brasil, 2007). O programa envolve a administração escolar e as equipes locais de Atenção Primária à Saúde, que atuam como parceiras. De acordo com as diretrizes do Ministério da Saúde, as atividades devem ser definidas levando em conta as necessidades da comunidade e a realidade do local, para que os serviços oferecidos respondam a essas demandas (Brasil, 2025).

A saúde mental, escolhida como tema para este projeto, é relevante para a vida dos estudantes. Segundo a Organização Mundial da Saúde, uma em cada sete pessoas entre 10 e 19 anos apresenta algum transtorno mental no mundo. Além disso, a organização aponta a importância do apoio das famílias, das escolas e das comunidades, considerando fatores como violência, discriminação e dificuldades econômicas (World Health Organization, 2025).

A saúde mental é apenas um aspecto do PSE, que abrange uma variedade de atividades voltadas ao bem-estar dos alunos, como alimentação saudável, prática de exercícios físicos, saúde bucal e prevenção das violências. Segundo o Ministério da Saúde, a promoção da saúde mental está entre os temas prioritários do ciclo 2025–2026 (Brasil, 2025).

Assim, este projeto poderia incluir rodas de conversa ou jogos em grupo como exemplos de atividades planejadas, mantendo o cadastro aberto a outros temas do PSE. Também é importante acompanhar e avaliar o andamento das atividades. Segundo o Ministério da Saúde, o monitoramento pode ajudar a identificar pontos de melhoria e oferecer suporte às equipes gestoras (Brasil, [s. d.]).

Foi identificado o seguinte problema: a equipe fictícia CodeSaúde mantém os registros das atividades em anotações, formulários e planilhas separados. Por isso, será planejada uma aplicação em C, executada em um terminal, que reúna dados sobre as ações, como escola, tema, data prevista e quantidade de participantes. Este trabalho apresenta o problema, os requisitos e os algoritmos do sistema, que utilizará somente informações fictícias e coletivas.

## **2 Problema, objetivos e justificativa**

### **2.1 Problema e questão de pesquisa**

A partir do cenário descrito, verifica-se que a equipe teria a necessidade de consultar diversos registros para conhecer as ações planejadas, realizadas e a participação dos alunos, uma vez que essas informações não estão centralizadas. O problema aqui discutido é como organizar esses registros usando os conhecimentos adquiridos durante o segundo semestre.

Como uma solução desenvolvida em linguagem C pode ajudar a equipe CodeSaúde a planejar e controlar ações coletivas de saúde nas escolas participantes do PSE, preservando a privacidade dos alunos?

### **2.2 Objetivo geral**

Construir uma aplicação em linguagem C que auxilie a equipe escolar e a equipe de saúde a planejar, monitorar e acompanhar as ações do PSE, usando dados fictícios das atividades coletivas.

### **2.3 Objetivos específicos**

- a) Determinar os dados necessários para o planejamento e o monitoramento das ações.
- b) Estabelecer os requisitos e os limites desse sistema de informação.
- c) Desenvolver fluxogramas e pseudocódigos para definir o funcionamento do sistema.
- d) Criar funções relacionadas ao cadastro, à listagem, à consulta, à atualização e à geração de resumo das ações.
- e) Elaborar um plano de testes para determinar as verificações a serem realizadas.

### **2.4 Justificativa acadêmica e social**

Este projeto integra conhecimentos de produção textual, lógica de programação e desenvolvimento de software em um único objetivo. Ele envolve, entre outros aspectos, a conversão dos requisitos em uma solução que possa ser usada pela equipe. Isso significa que tarefas de programação, como escrever algoritmos e funções e usar vetores, estruturas, decisões e repetições, devem ser realizadas com base nos requisitos definidos. Dessa forma, torna-se mais fácil avaliar o quanto o desenvolvimento está se aproximando da ideia originalmente proposta.

Quanto aos impactos sociais, o programa poderia ajudar no planejamento de atividades voltadas à saúde dos alunos. A avaliação do protótipo, porém, ficará concentrada na organização das ações e das informações. Trata-se de uma solução tecnológica de apoio à equipe escolar e de saúde, que não realiza atendimento terapêutico nem permite avaliar mudanças nas condições de saúde dos alunos.

### 3 Usuários e delimitação da solução

#### 3.1 Usuários

A equipe CodeSaúde será o cliente fictício deste projeto. O aplicativo será usado em um computador, com um operador por vez. Nesta fase, os usuários terão as mesmas opções, sem contas, senhas ou permissões diferentes.

| Usuário                  | Necessidades                                                                                           |
|--------------------------|--------------------------------------------------------------------------------------------------------|
| Coordenação da CodeSaúde | Consultar a lista de ações, acompanhar sua situação e ver o resumo de participação.                    |
| Equipe pedagógica        | Registrar a unidade escolar, o público-alvo, a data e o responsável definidos para as ações conjuntas. |
| Equipe de saúde          | Consultar as ações definidas e atualizá-las quanto à realização e à quantidade de participações.       |

Alunos, pais e a comunidade escolar são beneficiários indiretos das ações coletivas. Eles não precisarão acessar o protótipo nem fornecer informações pessoais para que o programa funcione. Os professores poderão utilizar o sistema como integrantes da equipe pedagógica.

#### 3.2 Escopo do aplicativo

O PSE em Ação terá opções para registrar ações, visualizá-las, pesquisá-las por código, unidade escolar ou tema, atualizar sua situação e gerar um resumo geral. Os dados de cada ação ficarão em um único registro, permitindo atualizar sua situação e a participação efetiva. O tema usado na demonstração será a saúde mental, mas outras ações do PSE também poderão ser cadastradas.

As informações serão armazenadas em um vetor com, no máximo, 100 ações. Ao encerrar o programa, os registros serão descartados e a memória será liberada. Essa abordagem permite trabalhar dentro da proposta inicial, dispensando banco de dados, conexão com a internet e elementos visuais avançados.

#### 3.3 Limitações e confidencialidade

Como se trata de um protótipo, os exemplos da demonstração serão restritos a escolas, responsáveis pelas atividades e públicos fictícios. Serão registradas quantidades de participações, sem incluir nomes de alunos, diagnósticos, históricos médicos, relatos individuais ou outros dados que permitam identificar os estudantes. Assim, o público-alvo indicará um grupo, como “alunos do 8º ano”, e o responsável será representado por uma função ou equipe fictícia.

A estrutura apresentada visa auxiliar na gestão das ações coletivas da escola. Consultas individuais, busca ou avaliação de psicólogos, cadastro de pacientes, triagem e orientação de tratamento não fazem parte desse escopo. Além disso, o sistema não terá integração com prontuários eletrônicos ou sistemas oficiais de saúde.

## 4 Requisitos do sistema

### 4.1 Requisitos funcionais

Os requisitos funcionais descrevem as operações previstas. Os critérios de aceitação indicam o comportamento que deverá ser verificado na implementação.

| ID   | Requisito                                                    | Critério de aceitação                                                                                             |
|------|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| RF01 | Cadastrar uma ação com os campos definidos na seção 5.       | Gravar um código único, os dados obrigatórios e a situação inicial planejada; confirmar o cadastro.               |
| RF02 | Listar todas as ações cadastradas.                           | Exibir os campos de cada registro em blocos identificados; avisar quando não houver ações.                        |
| RF03 | Pesquisar ações por código, escola ou tema.                  | Localizar o código exato ou todos os registros que contenham o texto pesquisado; informar ausência de resultados. |
| RF04 | Atualizar a situação para planejada, realizada ou cancelada. | Localizar a ação pelo código e alterar somente o registro correspondente.                                         |
| RF05 | Registrar a participação efetiva de uma ação realizada.      | Solicitar quantidade válida ao marcar realizada; manter zero nas demais situações.                                |
| RF06 | Gerar um resumo geral.                                       | Exibir os totais por situação, as participações efetivas e a comparação com o previsto, conforme a seção 5.2.     |
| RF07 | Validar as entradas e a capacidade de armazenamento.         | Recusar códigos duplicados, campos vazios, datas inválidas, quantidades negativas e cadastro além de 100 ações.   |
| RF08 | Manter o menu disponível até o encerramento.                 | Retornar ao menu após cada operação; informar opções inválidas e encerrar somente pela opção 0.                   |

A opção de atualização também permitirá corrigir a quantidade efetiva de uma ação já realizada. Os demais dados do cadastro e o código permanecerão fixos nesta versão.

## 4.2 Qualidade e regras de funcionamento

| ID    | Requisito não funcional                                                  | Como verificar                                                                                             |
|-------|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| RNF01 | Executar em terminal, em linguagem C, com recursos da biblioteca padrão. | Compilar e executar no ambiente indicado pelo professor, sem dependências externas.                        |
| RNF02 | Separar as responsabilidades em funções e utilizar struct e vetor.       | Revisar a organização do código; a função main deve coordenar o menu e chamar as operações.                |
| RNF03 | Apresentar mensagens em português e orientar a correção de erros.        | Verificar se cada entrada inválida gera uma explicação e nova oportunidade de preenchimento.               |
| RNF04 | Manter os dados somente em memória e comportar até 100 ações.            | Cadastrar até o limite, recusar a 101ª ação e verificar que uma nova execução começa sem registros.        |
| RNF05 | Restringir o modelo a informações coletivas e dados fictícios.           | Revisar os campos, os exemplos e a documentação para garantir ausência de dados individuais de estudantes. |

## 4.3 Regras de funcionamento

- R01** Toda nova ação começa como planejada, com participação efetiva igual a zero. O código deve ser um inteiro entre 1 e 999999 e não pode se repetir.
- R02** As quantidades prevista e efetiva devem ser inteiros entre 0 e 10000. O limite superior é uma escolha do protótipo para controlar as entradas.
- R03** Textos obrigatórios não podem conter apenas espaços nem ultrapassar o tamanho do campo. A data deve existir no calendário e seguir o formato DD/MM/AAAA, com ano entre 1900 e 9999.
- R04** Ao alterar uma ação realizada para planejada ou cancelada, a quantidade efetiva passa a zero. Antes da confirmação, o sistema informa essa consequência.
- R05** A pesquisa por escola ou tema procura trechos do texto e diferencia letras maiúsculas, minúsculas e acentos. O menu de pesquisa deve explicar esse comportamento.
- R06** O total efetivo pode superar o previsto. Nessa situação, o percentual poderá ser maior que 100%, sem que isso seja tratado como erro.

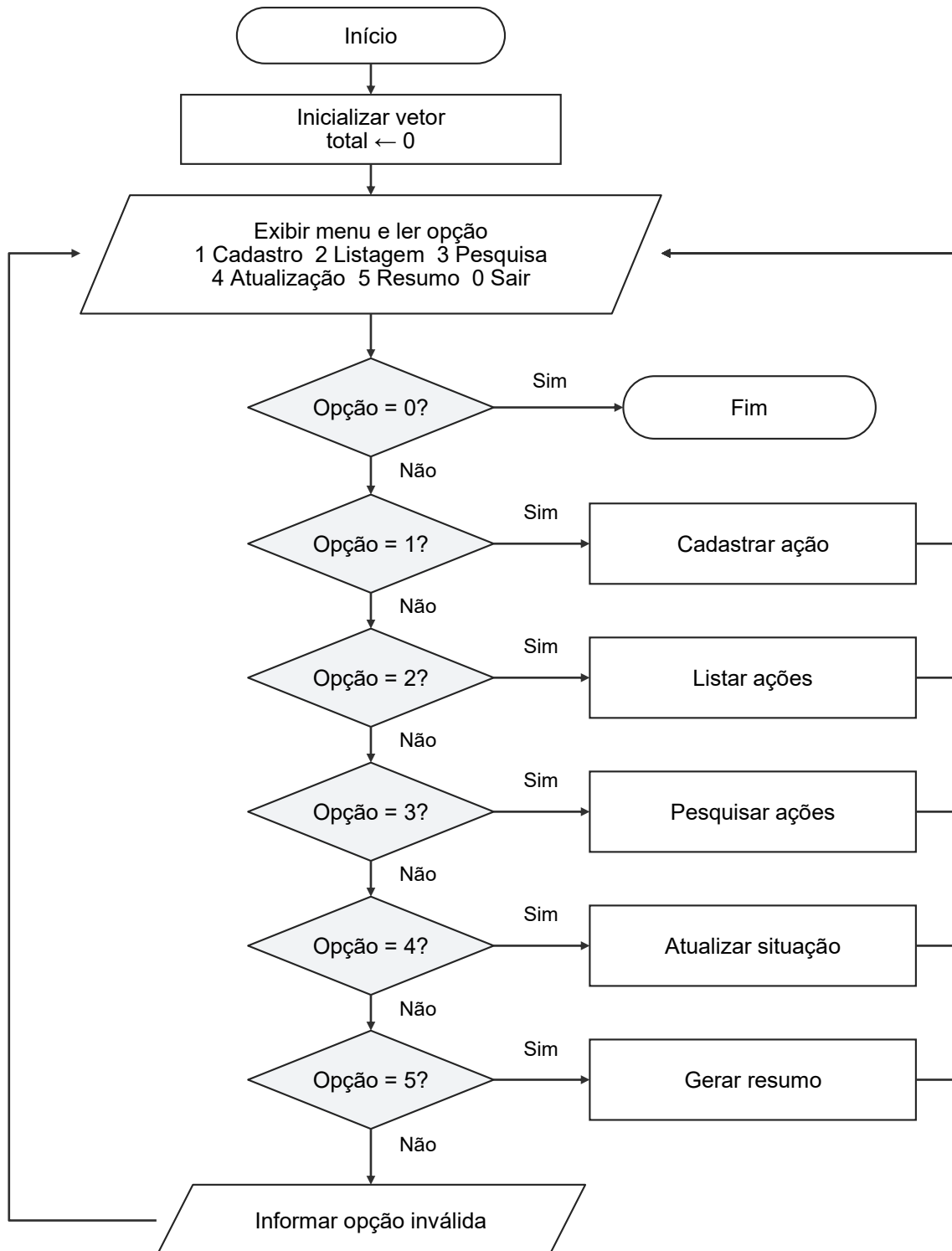
## 5. Estrutura de cada ação

A implementação utilizará uma struct `Acao`, armazenada no vetor `acoes[100]`. A variável `total` indicará quantas posições estão ocupadas. Os tamanhos de texto abaixo incluem o caractere de término da string.

| Campo                     | Tipo previsto em C    | Conteúdo                                      |
|---------------------------|-----------------------|-----------------------------------------------|
| <code>codigo</code>       | <code>int</code>      | Identificador único da ação.                  |
| <code>escola</code>       | <code>char[81]</code> | Nome fictício da escola.                      |
| <code>tema</code>         | <code>char[81]</code> | Tema da ação, como saúde mental ou vacinação. |
| <code>dataPrevista</code> | <code>char[11]</code> | Data no formato DD/MM/AAAA.                   |
| <code>publicoAlvo</code>  | <code>char[81]</code> | Grupo participante, sem nomes individuais.    |
| <code>responsavel</code>  | <code>char[81]</code> | Equipe ou função fictícia responsável.        |
| <code>previstos</code>    | <code>int</code>      | Quantidade esperada de participações.         |
| <code>situacao</code>     | <code>int</code>      | 1 = planejada; 2 = realizada; 3 = cancelada.  |
| <code>efetivos</code>     | <code>int</code>      | Participações registradas após a realização.  |

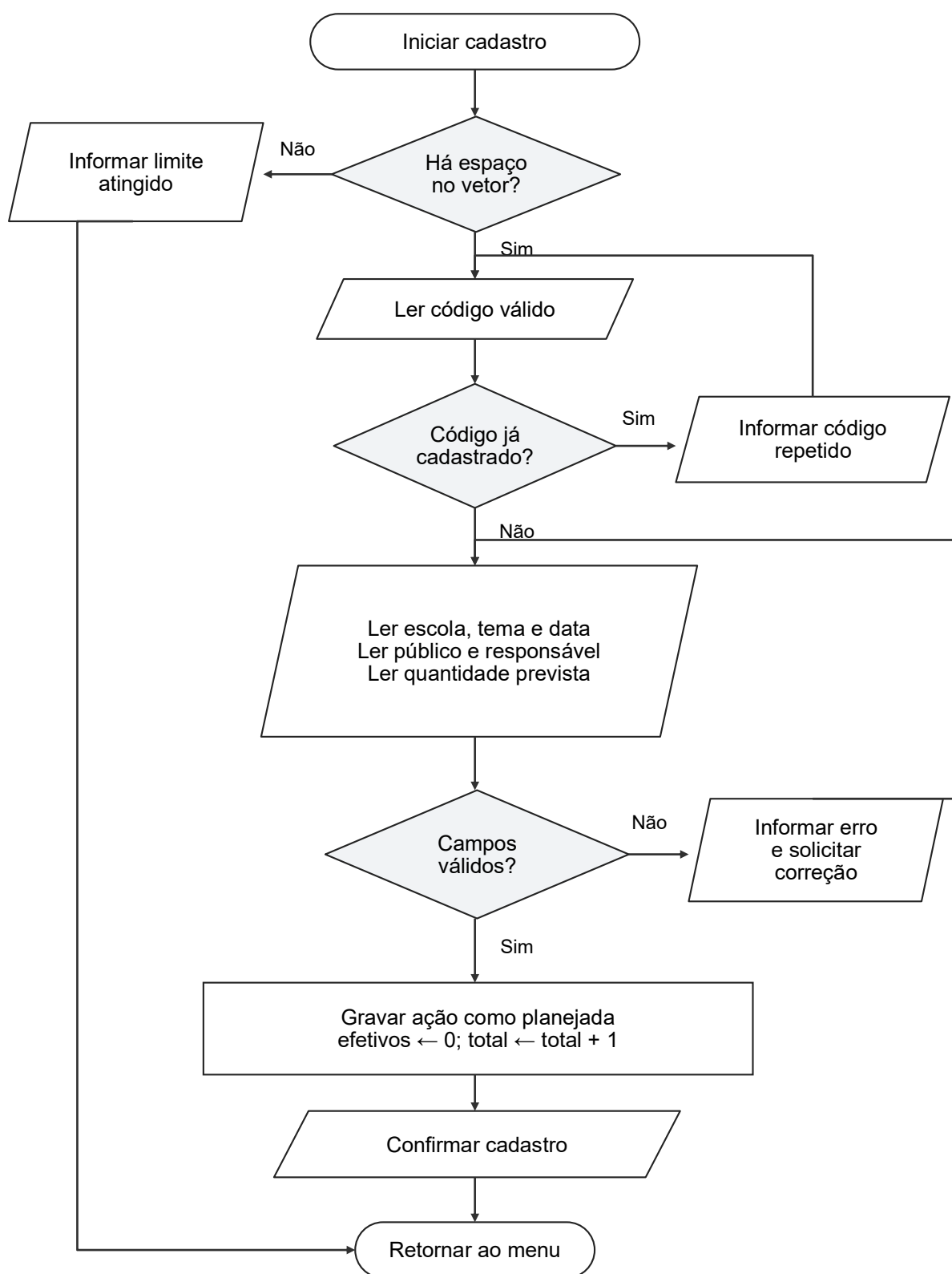
## 6 Fluxogramas

### 6.1 Fluxo geral do sistema



Fonte: elaboração própria (2026). Formas arredondadas indicam início ou fim; retângulos, processamento; paralelogramos, entrada ou saída; losangos, decisão. As setas à direita retornam ao menu.

## 6.2 Fluxo detalhado do cadastro



Fonte: elaboração própria (2026). A validação dos campos é resumida no losango central. Na implementação, cada campo inválido será solicitado novamente, conforme os procedimentos da seção 7.

## 7 Pseudocódigos

### 7.1 Menu principal e localização por código

Os algoritmos usam índices a partir de zero. O vetor é compartilhado com as funções; Cadastrar devolve a nova quantidade de registros. LerInteiro e LerTexto repetem a leitura até receber uma entrada válida, conforme a seção 7.2.

```
ALGORITMO Principal
    CONSTANTE CAPACIDADE ← 100
    DECLARAR acoes[CAPACIDADE] DO TIPO Acao
    total ← 0
    REPITA
        ESCREVER "1 Cadastrar | 2 Listar | 3 Pesquisar"
        ESCREVER "4 Atualizar | 5 Resumo | 0 Sair"
        opcao ← LerInteiro("Opção", 0, 5)
        ESCOLHA opcao
            CASO 1: total ← Cadastrar(acoes, total)
            CASO 2: Listar(acoes, total)
            CASO 3: Pesquisar(acoes, total)
            CASO 4: Atualizar(acoes, total)
            CASO 5: Resumir(acoes, total)
            CASO 0: ESCREVER "Encerrando. Os dados serão descartados."
        FIM ESCOLHA
    ATÉ opcao = 0
FIM ALGORITMO

FUNÇÃO LocalizarCodigo(acoes, total, codigo)
    PARA i ← 0 ATÉ total - 1 FAÇA
        SE acoes[i].codigo = codigo ENTÃO
            RETORNAR i
    FIM SE
    FIM PARA
    RETORNAR -1
FIM FUNÇÃO

PROCEDIMENTO Listar(acoes, total)
    SE total = 0 ENTÃO
        ESCREVER "Nenhuma ação cadastrada."
        RETORNAR
    FIM SE
    PARA i ← 0 ATÉ total - 1 FAÇA
        ExibirAcao(acoes[i])
    FIM PARA
FIM PROCEDIMENTO
```

### Saída de cada registro

ExibirAcao apresenta código, escola, tema, data prevista, público-alvo, responsável, previstos, situação por extenso e efetivos. Os rótulos serão mantidos em todas as consultas para facilitar a leitura. Quando total for zero, o laço de localização não executará nenhuma iteração.

## 7.2 Validação e cadastro

```
FUNÇÃO LerInteiro(mensagem, minimo, maximo)
  REPITA
    ESCRIVER mensagem
    LER uma linha completa e retirar espaços das extremidades
    valido ← linha representa somente um inteiro entre minimo e maximo
    SE NÃO valido ENTÃO ESCRIVER "Informe um inteiro no intervalo."
  ATÉ valido
  RETORNAR valor inteiro convertido
FIM FUNÇÃO
```

```
FUNÇÃO LerTexto(mensagem, capacidade)
  REPITA
    ESCRIVER mensagem
    LER uma linha completa e retirar espaços das extremidades
    valido ← texto não vazio E tamanho em bytes < capacidade
    SE NÃO valido ENTÃO ESCRIVER "Texto vazio ou muito longo."
  ATÉ valido
  RETORNAR texto
FIM FUNÇÃO
```

```
FUNÇÃO Cadastrar(acoes, total)
  SE total = 100 ENTÃO
    ESCRIVER "Limite de 100 ações atingido."
    RETORNAR total
  FIM SE
  REPITA
    nova.codigo ← LerInteiro("Código", 1, 999999)
    repetido ← LocalizarCodigo(acoes, total, nova.codigo) <> -1
    SE repetido ENTÃO ESCRIVER "Código já cadastrado."
  ATÉ NÃO repetido
  nova.escola ← LerTexto("Escola fictícia", 81)
  nova.tema ← LerTexto("Tema da ação", 81)
  REPITA
    nova.dataPrevista ← LerTexto("Data DD/MM/AAAA", 11)
    valida ← DataValida(nova.dataPrevista)
    SE NÃO valida ENTÃO ESCRIVER "Data inválida."
  ATÉ valida
  nova.publicoAlvo ← LerTexto("Público coletivo", 81)
  nova.responsavel ← LerTexto("Equipe responsável fictícia", 81)
  nova.previdos ← LerInteiro("Participações previstas", 0, 10000)
  nova.situacao ← 1
  nova.efetivos ← 0
  acoes[total] ← nova
  ESCRIVER "Ação cadastrada como planejada."
  RETORNAR total + 1
FIM FUNÇÃO
```

DataValida verifica os dez caracteres do formato, as barras, os dígitos, o intervalo do ano, o mês e os dias existentes. Fevereiro tem 29 dias quando o ano é divisível por 400 ou quando é divisível por 4 e não por 100. A leitura completa deve descartar eventual excesso da linha para não contaminar a próxima entrada.

### 7.3 Pesquisa e atualização

```
PROCEDIMENTO Pesquisar(acoes, total)
  SE total = 0 ENTÃO
    ESCRIVER "Nenhuma ação cadastrada."
    RETORNAR
  FIM SE
  tipo ← LerInteiro("1 Código | 2 Escola | 3 Tema", 1, 3)
  SE tipo = 1 ENTÃO
    codigo ← LerInteiro("Código", 1, 999999)
    indice ← LocalizarCodigo(acoes, total, codigo)
    SE indice = -1 ENTÃO ESCRIVER "Ação não encontrada."
    SENÃO ExibirAcao(acoes[indice])
    FIM SE
  SENÃO
    ESCRIVER "A pesquisa diferencia maiúsculas e acentos."
    termo ← LerTexto("Trecho de escola ou tema", 81)
    encontrados ← 0
    PARA i ← 0 ATÉ total - 1 FAÇA
      SE tipo = 2 ENTÃO campo ← acoes[i].escola
      SENÃO campo ← acoes[i].tema
      FIM SE
      SE campo contém termo ENTÃO
        ExibirAcao(acoes[i])
        encontrados ← encontrados + 1
      FIM SE
    FIM PARA
    SE encontrados = 0 ENTÃO ESCRIVER "Nenhum resultado."
  FIM SE
FIM PROCEDIMENTO

PROCEDIMENTO Atualizar(acoes, total)
  codigo ← LerInteiro("Código da ação", 1, 999999)
  indice ← LocalizarCodigo(acoes, total, codigo)
  SE indice = -1 ENTÃO
    ESCRIVER "Ação não encontrada."
    RETORNAR
  FIM SE
  ExibirAcao(acoes[indice])
  novaSituacao ← LerInteiro("1 Planejada | 2 Realizada | 3 Cancelada",
    1, 3)
  SE novaSituacao = 2 ENTÃO
    novosEfetivos ← LerInteiro("Participações efetivas", 0, 10000)
  SENÃO
    novosEfetivos ← 0
    ESCRIVER "A quantidade efetiva será definida como zero."
  FIM SE
  confirmar ← LerInteiro("Confirmar alteração? 1 Sim | 0 Não", 0, 1)
  SE confirmar = 1 ENTÃO
    acoes[indice].situacao ← novaSituacao
    acoes[indice].efetivos ← novosEfetivos
    ESCRIVER "Ação atualizada."
  SENÃO
    ESCRIVER "Alteração não confirmada. Registro preservado."
  FIM SE
FIM PROCEDIMENTO
```

## 7.4 Resumo e cenário de verificação

```
PROCEDIMENTO Resumir(acoas, total)
  planejadas ← 0; realizadas ← 0; canceladas ← 0
  previstosGeral ← 0; previstosRealizadas ← 0; efetivos ← 0
  PARA i ← 0 ATÉ total - 1 FAÇA
    previstosGeral ← previstosGeral + acoes[i].previstos
    ESCOLHA acoes[i].situacao
      CASO 1: planejadas ← planejadas + 1
      CASO 2:
        realizadas ← realizadas + 1
        previstosRealizadas ← previstosRealizadas + acoes[i].previstos
        efetivos ← efetivos + acoes[i].efetivos
      CASO 3: canceladas ← canceladas + 1
    FIM ESCOLHA
  FIM PARA
  ESCREVER total, planejadas, realizadas, canceladas, com rótulos
  ESCREVER previstosGeral, previstosRealizadas, efetivos, com rótulos
  SE previstosRealizadas > 0 ENTÃO
    percentual ← 100.0 * efetivos / previstosRealizadas
    ESCREVER "Participação nas realizadas: ", percentual, "%"
  SENÃO
    ESCREVER "Percentual não calculável: previsão igual a zero."
  FIM SE
FIM PROCEDIMENTO
```

### Exemplo fictício de resultado esperado

O cenário abaixo será utilizado na etapa de testes. Ele representa dados simulados e resultados esperados, sem constituir registro de atividades reais ou teste já executado.

| Código | Tema         | Situação  | Previstos | Efetivos |
|--------|--------------|-----------|-----------|----------|
| 101    | Saúde mental | Planejada | 40        | 0        |
| 102    | Saúde mental | Realizada | 30        | 24       |
| 103    | Vacinação    | Realizada | 20        | 22       |
| 104    | Saúde bucal  | Cancelada | 25        | 0        |

Saída esperada: 4 ações, sendo 1 planejada, 2 realizadas e 1 cancelada. Total previsto geral: 115; previsto nas realizadas: 50; participações efetivas: 46; percentual de participação: 92%.

Com o vetor vazio, os totais serão zero e o percentual não será calculado. Alterar a ação 102 para cancelada deverá retirar suas 24 participações do resumo. A ação 103 verifica que a presença acima da previsão é permitida.

## 8 Processo e planejamento das sprints

O desenvolvimento seguirá um processo incremental, organizado em sprints de uma semana. Ao início de cada sprint, o grupo distribuirá as tarefas; ao final, revisará o que foi produzido e registrará as pendências. Cada funcionalidade será incorporada à versão anterior para facilitar a identificação de erros.

O planejamento considera oito semanas relativas ao início do projeto. As datas serão ajustadas ao calendário informado pelos professores, pois o enunciado não apresenta os prazos preenchidos. As duas primeiras sprints concentram os artefatos desta entrega; as demais organizam a implementação e a apresentação final.

| Sprint | Foco                         | Atividades e entrega verificável                                                                                          |
|--------|------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| 1      | Levantamento                 | Ler o desafio, consultar fontes, definir problema, usuários e escopo; criar o repositório e a identificação do grupo.     |
| 2      | Modelagem e primeira entrega | Revisar requisitos, riscos, fluxogramas e pseudocódigos; publicar o documento em docs e revisar o README da raiz.         |
| 3      | Base do sistema              | Implementar struct, vetor, menu e cadastro; verificar código duplicado, campos obrigatórios e limite de registros.        |
| 4      | Consultas                    | Implementar listagem e pesquisa por código, escola e tema; verificar busca sem resultado e múltiplas correspondências.    |
| 5      | Acompanhamento               | Implementar atualização, efetivos e resumo; conferir mudanças de situação, soma e divisão por zero.                       |
| 6      | Integração e testes          | Executar os cenários de validação, corrigir defeitos e registrar entradas, saídas esperadas e saídas obtidas.             |
| 7      | Documentação final           | Revisar requisitos e algoritmos; elaborar casos de uso, manual, instruções de execução e capturas de tela reais.          |
| 8      | Revisão e apresentação       | Conferir execução no ambiente da disciplina, organizar o GitHub e gravar vídeo de até 7 minutos com todos os integrantes. |

As responsabilidades serão alternadas entre programação, documentação e revisão, conforme a divisão registrada pelo grupo. Uma tarefa será considerada concluída quando o material estiver compreensível, revisado por outro integrante e incluído no repositório. Nas sprints de implementação, também deverá executar os cenários previstos para a funcionalidade.

## 9 Riscos e preparação dos testes

Os riscos abaixo orientam o acompanhamento do projeto. As classificações são estimativas de planejamento e deverão ser revistas durante as sprints.

| Risco                                               | Probabilidade e impacto | Resposta prevista                                                                                            |
|-----------------------------------------------------|-------------------------|--------------------------------------------------------------------------------------------------------------|
| Ampliação do escopo para consultas e dados clínicos | Média / alto            | Conferir cada nova ideia com os requisitos da AEP e manter o cadastro de ações.                              |
| Atraso por dificuldades em C                        | Alta / alto             | Implementar por etapas e revisar dúvidas a cada semana; priorizar as operações obrigatórias.                 |
| Erros de entrada ou cálculo                         | Alta / alto             | Centralizar a validação e conferir exemplos com resultado calculado previamente, incluindo divisão por zero. |
| Versões conflitantes ou perda de trabalho           | Média / alto            | Registrar alterações pequenas no GitHub e combinar a responsabilidade por cada arquivo antes da edição.      |
| Dados inadequados nos exemplos                      | Baixa / alto            | Revisar o conjunto de demonstração e manter apenas informações coletivas e fictícias.                        |

### 9.1 Verificações previstas

Os testes serão manuais e utilizarão dados fictícios. Para cada cenário, o grupo registrará a entrada, o resultado esperado, o resultado obtido e eventuais correções. A implementação ainda será realizada; portanto, esta seção descreve um plano de verificação.

- Cadastrar uma ação válida e tentar repetir o código; o segundo cadastro deverá ser recusado sem alterar o primeiro.
- Informar texto vazio, somente espaços, excesso de caracteres, número negativo, letras em campo numérico e opção de menu inexistente; o sistema deverá solicitar correção.
- Testar uma data impossível, como 31/02/2026, e anos bissextos; aceitar apenas datas existentes.
- Pesquisar um código inexistente e um termo presente em várias ações; verificar a mensagem de ausência e a exibição de todos os resultados.
- Realizar, cancelar e replanejar ações; conferir a confirmação da alteração e os efeitos nos totais de participação.
- Executar o exemplo da seção 7.4, o resumo vazio, o limite de 100 registros e uma nova execução após o encerramento.

## Referências

BRASIL. **Decreto nº 6.286, de 5 de dezembro de 2007**. Institui o Programa Saúde na Escola – PSE, e dá outras providências. Brasília, DF: Presidência da República, 2007. Disponível em: [https://www.planalto.gov.br/ccivil\\_03/\\_ato2007-2010/2007/decreto/d6286.htm](https://www.planalto.gov.br/ccivil_03/_ato2007-2010/2007/decreto/d6286.htm). Acesso em: 9 set. 2026.

BRASIL. Ministério da Saúde. **Perguntas frequentes (FAQ)**. Brasília, DF: Ministério da Saúde, 2025. Atualizado em: 3 jun. 2025. Disponível em: <https://www.gov.br/saude/pt-br/composicao/saps/pse/faq>. Acesso em: 9 set. 2026.

BRASIL. Ministério da Saúde. **Programa Saúde na Escola (PSE)**. Brasília, DF: Ministério da Saúde, [s. d.]. Disponível em: <https://www.gov.br/saude/pt-br/composicao/saps/pse>. Acesso em: 9 set. 2026.

WORLD HEALTH ORGANIZATION. **Mental health of adolescents**. Geneva: WHO, 2025. Publicado em: 1 set. 2025. Disponível em: <https://www.who.int/news-room/fact-sheets/detail/adolescent-mental-health>. Acesso em: 9 set. 2026.